UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2014/2015

SEPTEMBER EXAMINATION

UECS3253 WIRELESS APPLICATION DEVELOPMENT

THURSDAY, 18 SEPTEMBER 2014 TIME : 9.00 AM – 11.00 AM (2 HOURS)

BACHELOR OF ENGINEERING (HONS)
ELECTRONICS (COMPUTER NETWORKING)
BACHELOR OF SCIENCE (HONS) SOFTWARE ENGINEERING

Instruction to Candidates :

This Examination Paper consists of **FIVE (5)** questions and is divided into **TWO(2)** sections.

Section A: (Q1 & Q2) [Total: 50 marks]

Q1 and Q2 are compulsory.

Section B: (Q3 to Q5) [Total: 50 marks]

Answer **ANY TWO (2)** out of the **THREE (3)** questions given in this section. Each question carries 25 marks.

If more than two questions are answered in Section B, **only the first TWO (2)** answers will be marked and taken into account.

UECS3253 WIRELESS APPLICATION DEVELOPMENT

SECTION A: Answer ALL questions.

Q1. (a) Elaborate on the differences of mobile *native application* and mobile *Web application* running on Android devices from the following perspectives:

(i) Programming model (programming language and tool for design and development)
(ii) Ability to access to device's features
(iii) The manner in which programs are compiled.
(iv) Method of deployment to user device
(v) Software updates approach
(vi) TWO (2) advantages over the other respectively

(6 marks)

(b) (i) Specify the Java package which you need to import for writing code involving the Intent object. (1 mark)

(ii) Illustrate the differences between an *explicit* intent and *intrinsic* intent when you would like to switch from Activity1 to Activity2 by clicking on a button on Activity1 to allow a user to browse a Website http://www.google.com. Provide your answer with the help of the related Java source code snippets, AndroidManifest.xml AND UI layout XML language constructs. (10 marks)

Hint:

- You should specify the roles of the Intent object and <intent-filter> in your answer for both cases (*explicit* and *implicit* intents).
- You may assume your own valid names of parameters such as filenames, class names etc. if not mentioned in the question.
- In both cases (*explicit* and *implicit* intents), we assume Activity2 has been compiled and installed in the mobile device.

(c) (i) The mobile Android application publishing process consists of two major phases, i.e. *preparation of application for release* and the *actual release of application.*

Describe FOUR (4) tasks and activities involved in the processes.

(4 marks)

(ii) You are required to configure the necessary application setting in the AndroidManifest.xml file. Describe FOUR (4) types of information you must specify in the AndroidManifest.xml for your mobile application to be "release ready", if you are releasing a mobile application which allows users to perform chatting over a network. Write the relevant XML language construct, if necessary to illustrate your case. (4 marks)

[Total : 25 marks]

Q2. Two of the various activities of an Android app are named **MeowActivity** and **QuackActivity** respectively. The MeowActivity defines the interface from the XML layout file named **meow.xml** while the QuackActivity defines the interface from the XML layout file named **quack.xml**.

Figure 2.1 below shows the MeowActivity when resumed while Figure 2.2 shows the MeowActivity after a user enters values using the various views. Figure 2.3 shows the QuackActivity that is started when a user taps on the **Next** button in the MeowActivity.

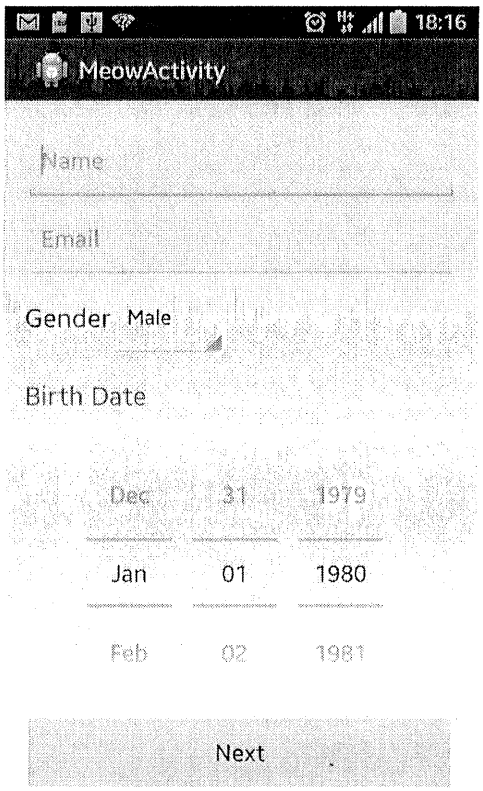

Figure 2.1

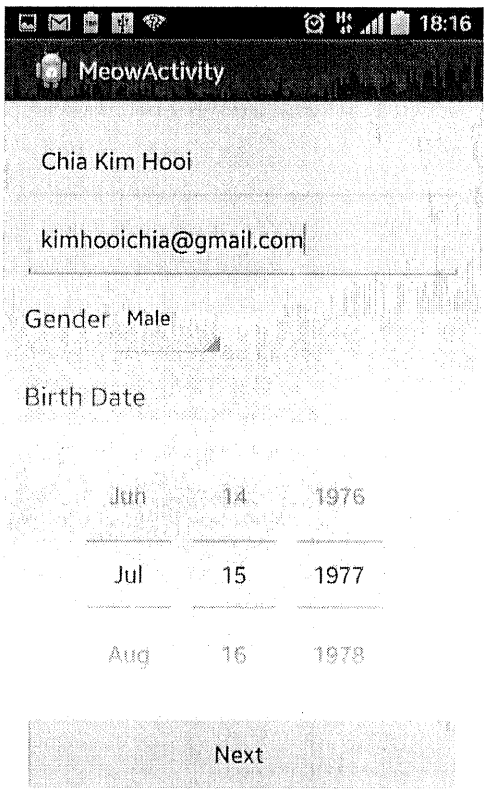

Figure 2.2

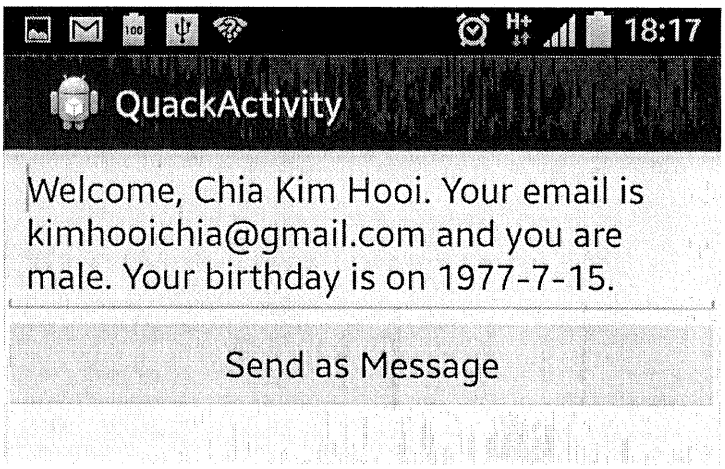

Figure 2.3

Q2. (Continued)

(a) Complete the XML code below for the layout file **meow.xml**. When the **Next** button is tapped by a user, the method *goQuack* will be invoked. You must ensure that each view is assigned with an appropriate **id**. Assume that the required string resources have been defined in the **strings.xml** file. You may use any appropriate names for the string resources in your answer.

HINT: The two **EditText** views for inputting name and email have been defined for you. You DO NOT have to specify the layout attributes for the views in your answer.

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/androi
d"
 xmlns:tools="http://schemas.android.com/tools"
 android:layout_width="match_parent"
 android:layout_height="match_parent">

<EditText
     android:id="@+id/et_name"
      android:layout_width="fill_parent"
      android:layout_height="60dp"
      android:hint="@string/field_name" />

<EditText
     android:id="@+id/et_email"
      android:layout_width="fill_parent"
      android:layout_height="60dp"
      android:layout_alignLeft="@id/et_name"
      android:layout_below="@id/et_name"
      android:inputType="textEmailAddress"
      android:hint="@string/field_email" />

    <!-- Write your answer for this section→

</RelativeLayout>
```

(7 marks)

(b) (i) Write the required Java code snippets to programmatically set the date of the **DatePicker** view in MeowActivity to the date January 1, 1980.
(2 marks)

(ii) When the *goQuack* method in MeowActivity is invoked, it starts QuackActivity and passes the **name**, **email**, **gender** and **birthdate** as extras to QuackActivity. Write the required Java code for the *goQuack* method. (7 marks)

Q2 (b). (Continued)

(iii) Write the required Java code snippets for the *onCreate* method of QuackActivity. When invoked, the method receives the extras from the intent and programmatically creates the interface as shown in Figure 2.3. The layout to be used is **LinearLayout**. The EditText view should be set with the text as shown in Figure 2.3 with values received from the intent. A user is allowed to edit the text in the EditText before sending.

Note: You DO NOT have to write the code to handle the *onClick* event for the Button view. (9 marks)

[Total : 25 marks]

SECTION B: Answer ONLY TWO (2) questions

Q3. (a) Propose the appropriate data storage technique(s) to be employed to tackle the following usage scenario of a mobile application.

Chong is engaged by a company to develop a mobile native e-promotion application as a freelance project. Based on the requirement specified by the client, the mobile e-Wallet system should have the following features:

- enable a mobile user to register as a member via mobile application
- enable a mobile user to view and update personal profile information
- enable a mobile user to store user personal preference data such as preferred background theme color, preferred category of promotions, current and recent locations etc.
- enable a mobile user to store the mobile setting data of the user elicited in the background such as the last login date/time, user mobile device specification (such as model, resolution, screen size etc.) etc.
- enable a mobile user to search, filter and view the merchants' promotion offerings for the credit or debit card(s) he is interested in
- enable a mobile user to rate and recommend promotion offerings
- enable a mobile user to export the search result to a file which can be downloaded for use offline or shared with other people

Chong consults you on the best way to implement such a system to persist data such as

- User personal preference data
- Registered user's personal profile information
- Mobile setting data of the user
- Rating and recommendation of promotion information
- Search results exported
- Merchant and promotion information

Provide Chong with your professional technical advice on the various data persistence options feasible in Android and briefly justify your proposal to fulfill the various data persistence needs of the targeted application.

(6 marks)

(b) A SQLite database table has been created as shown in Figure 3, where the table has three fields: **_id**, **name** and **email**:

Q3 (b). (Continued)

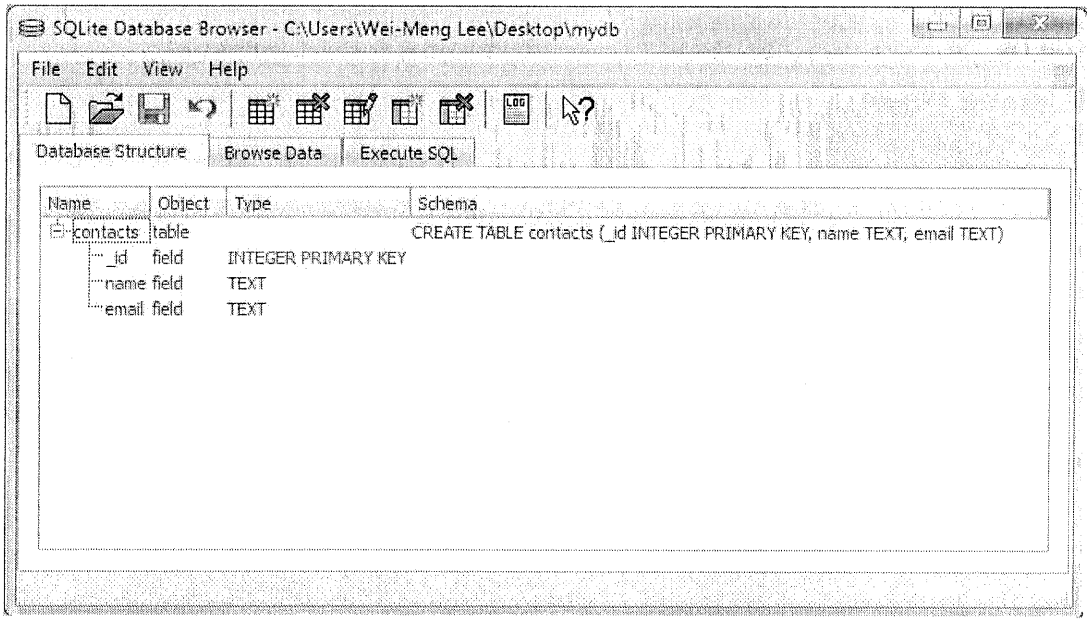

Figure 3

Code snippet of a database helper class is shown as follows:

```
public class DBAdapter {

...

     private static class DatabaseHelper extends
     SQLiteOpenHelper
     {
          DatabaseHelper(Context context)
          {
          super(context, DATABASE_NAME, null,
          DATABASE_VERSION);
          }
          @Override
          public void onCreate(SQLiteDatabase db)
          {
               try {
               db.execSQL(DATABASE_CREATE);
               } catch (SQLException e) {
               e.printStackTrace();
               }
          }
          ...
     }

     //---insert a contact into the database---
     public long insertContact(String name, String email)
```

Q3 (b). (Continued)

```
{
      ContentValues initialValues = new
      ContentValues();
      initialValues.put(KEY_NAME, name);
      initialValues.put(KEY_EMAIL, email);
      return db.insert(DATABASE_TABLE, null,
      initialValues);
}

//---deletes a particular contact---
public 8oolean deleteContact(long rowId)
{
      return db.delete(DATABASE_TABLE, KEY_ROWID +
      "=" + rowId, null) > 0;
}

//---retrieves all the contacts---
public Cursor getAllContacts()
{
      return db.query(DATABASE_TABLE, new String[]
      {KEY_ROWID, KEY_NAME,
      KEY_EMAIL}, null, null, null, null, null);
}

//---retrieves a particular contact---
public Cursor getContact(long rowId) throws
SQLException
{
      Cursor mCursor =
      db.query(true, DATABASE_TABLE, new String[]
      {KEY_ROWID,KEY_NAME, KEY_EMAIL}, KEY_ROWID +
      "=" + rowId, null,null, null, null, null);

      if (mCursor != null) {
            mCursor.moveToFirst();
      }
      return mCursor;
}

//---updates a contact---
public 8oolean updateContact(long rowId, String
name, String email)
{
      ContentValues args = new ContentValues();
      args.put(KEY_NAME, name);
      args.put(KEY_EMAIL, email);
```

Q3(b). (Continued)

```
            return db.update(DATABASE_TABLE, args,
            KEY_ROWID + "=" + rowId, null) > 0;
        }
        ...
}
```

Code snippet of DisplayContact method is shown as follows:

```
public void DisplayContact(Cursor c)
{
        Toast.makeText(this,
            "id: " + c.getString(0) + "\n" +
            "Name: " + c.getString(1) + "\n" +
            "Email: " + c.getString(2),
        Toast.LENGTH_LONG).show();
}
```

Write Java code snippets to perform the following:

(i) Add the following two contacts into the database.

_id	name	email
1	John Ting	jt124@gmail.com
2	Samuel Chung	sc123@gmail.com

(4 marks)

(ii) Retrieve all contacts you have just inserted in part (i). Also, import ONE (1) specific database related Java package for the codes to work. (5 marks)

(c) You are a member of a project team that is going to develop an Android countdown application. The application allows the user to set the name, date and time of the event. The app will display the event title and number of days, hours, minutes and seconds left to the event.

The event title, date and time are saved in a shared preference file named **countdownData**. The event title will be saved as a string using the key **eventTitle** while the date and time are saved as a long integer representing the number of milliseconds since January 1, 1970, 00:00:00.0 (Epoch time) using the key **eventDateTime**.

The following variables have been declared as shown in the code snippet below:

```
String eventTitle;
Calendar eventDateTime = Calendar.getInstance();
```

Q3 (c). (Continued)

(i) Assume that the variable **eventTitle** and **eventDateTime** contain the event title and event date and time respectively. Write the required code to save the event title and event date and time. (5 marks)

(ii) Write the required code to read the data saved in the **countdownData** shared preference file and assign the values to the **eventTitle** and **eventDateTime** variables. You may assume any appropriate default values to be used if the key does not exist. (5 marks)

[Total : 25 marks]

Q4. (a) A Login Form UI is shown in Figure 4 below.

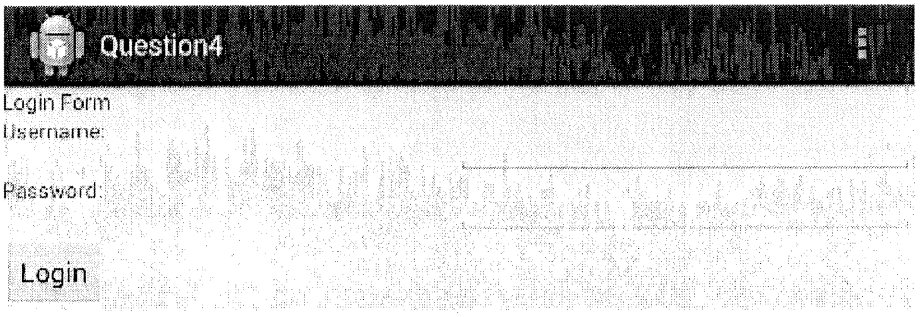

Figure 4

The values stored in the **res/values/strings.xml** file are as follows:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">Question4</string>
    <string name="title">Login Form</string>
    <string name="button_label">Login</string>
    <string name="password_label">Password:</string>
    <string name="username_label">Username:</string>
</resources>
```

(i) Synthesize the content of the XML layout file named "**main.xml**" to construct the UI shown in Figure Q4. A template to place your answer is provided to you as follows:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/andr
oid"
android:id="@+id/root"
android:orientation="__________"
android:layout_width="fill_parent"
```

Q4 (a)(i). (Continued)

```
        android:layout_height="wrap_content">

        <!-write your answer for this section -->

    </LinearLayout>
```

Hint: Avoid using physical dimension such as number of pixels to control the width and height of the view. Apply **layout_weight**, **fill_parent** or **wrap_content** values to control the width and height wherever appropriate instead. (12 marks)

(ii) Write the relevant Java method code snippet to load the XML layout at runtime. (2 marks)

(b) Suppose the **MainActivity** of an Android app displays a list of items in a ListView. Clicking on an item in the ListView starts an activity named **DisplayImageActivity** which displays an image using ImageView.

You are given the following code snippets:

MainActivity:

```
ArrayList<String>imgDesc = new ArrayList<String>();
ArrayList<String> imgFile = new ArrayList<String>();
ListView listView = (ListView)
findViewById(R.id.list_view);
```

DisplayImageActivity:

```
String file =
getIntent().getStringExtra(MainActivity.EXTRA_IMG_FILE);
Bitmap bitmap = BitmapFactory.decodeFile(file);
```

(i) Write the required code snippet for the **MainActivity** to populate the ListView **list_view** with the values of the ArrayList **imgDesc**. The layout file for the items of the ListView is **list_item**. (3 marks)

(ii) Write the required code for the **OnItemClickListener** handler in MainActivity for the above ListView. The corresponding image filename in the ArrayList **imgFile** will be passed to **DisplayImageActivity** as an extra. (5 marks)

(iii) Write the required Java code snippet to programmatically create an ImageView view to display the image specified in the **bitmap** object in **DisplayImageActivity**. (3 marks)

[Total : 25 marks]

Q5. (a) (i) In Android development, it is recommended to always perform network operations on a separate thread from the User Interface (UI) thread. Explain the reason for this recommendation. (3 marks)

(ii) If you run a synchronous method in a UI thread, your application will crash when it is run on a device running Android 3.0 and later.

Propose a solution to perform network operation on Android applications by mentioning the Java class and methods that are involved. (4 marks)

(iii) Demonstrate how they may be implemented by writing sample code snippet in Java to enable the following *DownloadImage* method to be performed on devices running Android 3.0 or later.

```
//Downloading an image from the internet
private Bitmap DownloadImage(String URL)
{
        Bitmap bitmap = null;
        InputStream in = null;

        try {
                in = OpenHttpConnection(URL);
                bitmap = BitmapFactory.decodeStream(in);
                in.close();
        } catch (IOException e1) {
                Log.d("NetworkingActivity", e1.getLocalizedMessage());
        }
        return bitmap;
}
```

(5 marks)

Hint: The parameter sent to the task upon execution is of type String and the type of the result of the background computation is a Bitmap.

(b) Suppose you are involved in a project to develop a Photo Gallery app for Android platform. The **MainActivity** of the app establishes an HTTP connection with a server to retrieve the required data to display a list of **albums**. Data is sent as a JSON Array in the example as shown in Figure 5.1

```
[
        {
                "id":"207",
                "albumName":"One Night in George Town, Penang",
                "albumPosted":"2014-06-21"
        },
        {
                "id":"197",
                "albumName":"One Weekend in Muar - Bandar Maharani",
```

```
            "albumPosted":"2014-05-10"
        },
        {
            "id":"188",
            "albumName":"Ocean Park Hong Kong",
            "albumPosted":"2013-11-30"
        },
        {
            "id":"186",
            "albumName":"I Love Hong Kong 2013",
            "albumPosted":"2013-11-27"
        }
]
```

Figure 5.1

Q5(b). (Continued)

(i) Complete the Java code snippet below which is used to establish an HTTP connection with the web server to obtain an **InputStream** object for the variable **in**. The code should use the appropriate classes of the **org.apache.http** package.

```
InputStream in = null;
String url = "http://ilove.hk/gallery/albums/json";
try {
        /*
        Write your answer for this section
        */
}
catch (ClientProtocolException ex) {}
catch (IOException ex) {}
```

(7 marks)

(ii) Suppose that **result** is a String that contains data in JSON format as shown in Figure 5.1. Complete the code snippet below to process the JSON string to fill the **albumNames** ArrayList of Strings with the values of the **albumName** property of each object in the JSON array.

```
ArrayList<String> albumNames = new ArrayList<String>();

/*
        Write your answer for this section
*/
```

(6 marks)
[Total : 25 marks]

APPENDIX

Activity

```
public class Activity extends ApplicationContext {

        public void setContentView (int layoutResID);
        public void setContentView (View view);
        public void setContentView (View view, ViewGroup.LayoutParams params);
        ...
}
```

Intent

```
public Intent ()
public Intent (String action)
public Intent (String action, Uri uri)
public Intent (Context packageContext, Class<?> cls)
public Intent (String action, Uri uri, Context packageContext, Class<?> cls)
```

Standard actions that Intent defines for launching activities (usually through startActivity(Intent):

- ACTION_MAIN
- ACTION_VIEW
- ACTION_ATTACH_DATA
- ACTION_EDIT
- ACTION_PICK
- ACTION_CHOOSER
- ACTION_GET_CONTENT
- ACTION_DIAL
- ACTION_CALL
- ACTION_SEND
- ACTION_SENDTO
- ACTION_ANSWER
- ACTION_INSERT
- ACTION_DELETE
- ACTION_RUN
- ACTION_SYNC
- ACTION_PICK_ACTIVITY
- ACTION_SEARCH
- ACTION_WEB_SEARCH
- ACTION_FACTORY_TEST

View

```
public final View findViewById (int id);
```

Shared Preferences

```
public interface SharedPreferences {
        public abstract SharedPreferences.Editor edit ();
        public abstract boolean getBoolean (String key, boolean defValue);
        public abstract float getFloat (String key, float defValue);
        public abstract int getInt (String key, int defValue);
        public abstract long getLong (String key, long defValue);
        public abstract String getString (String key, String defValue);
        ...
}

public static interface SharedPreferences.Editor {
        public abstract void apply ();
        public abstract SharedPreferences.Editor clear ();
        public abstract boolean commit ();
        public abstract SharedPreferences.Editor
```

```
        putBoolean (String key, boolean value);
        public abstract SharedPreferences.Editor putFloat (String key, float value);
        public abstract SharedPreferences.Editor putInt (String key, int value);
        public abstract SharedPreferences.Editor putLong (String key, long value);
        public abstract SharedPreferences.Editor
        putString (String key, String value);
        public abstract SharedPreferences.Editor remove (String key);
        ...
}
```

JSON

```
public class JSONArray extends Object {
        public JSONArray (String json);
        public JSONArray (Object array);
        public Object get (int index);
        public boolean getBoolean (int index);
        public double getDouble (int index);
        public int getInt (int index);
        public JSONArray getJSONArray (int index);
        public JSONObject getJSONObject (int index);
        public long getLong (int index);
        public String getString (int index);
        ...
}

public class JSONObject extends Object {
        public JSONObject (String json);
        public Object get (String name);
        public boolean getBoolean (String name);
        public double getDouble (String name);
        public int getInt (String name);
        public JSONArray getJSONArray (String name);
        public JSONObject getJSONObject (String name);
        public long getLong (String name);
        public String getString (String name);
        ...
}
```

HTTP

```
public class DefaultHttpClient extends AbstractHttpClient {
        public DefaultHttpClient ();
        public final HttpResponse execute (HttpUriRequest request);
        ...
}

public class HttpGet extends HttpRequestBase {
        public HttpGet (URI uri);
        public HttpGet (String uri);
        ...
}

public class HttpPost extends HttpEntityEnclosingRequestBase {
        public HttpPost (URI uri);
        public HttpPost (String uri);
        ...
}

public interface HttpResponse implements HttpMessage {
        public abstract StatusLine getStatusLine ();
        public abstract HttpEntity getEntity ();
        ...
}
```

```
public interface StatusLine {
	public abstract int getStatusCode ();
	...
}

public interface HttpEntity {
	public abstract InputStream getContent ();
	public abstract long getContentLength ();
	public abstract Header getContentType ();
	...
}
```

AsyncTask

```
public AsyncTask ()
{
	public final AsyncTask<Params, Progress, Result> execute (Params... params);
	protected void onPreExecute ();
	protected void onProgressUpdate (Progress... values);
	protected abstract Result doInBackground (Params... params);
	protected void onPostExecute (Result result);
	...
}
```